Final teórico de Paradigmas y Lenguajes

Modelos de examen y banco de preguntas · Ciclo 2026

Este documento reúne el modelo de final que circula de la cátedra, tres modelos nuevos armados con el mismo formato a partir de las diapositivas 2026, un banco de 100 preguntas ordenado por unidad y, al final, todas las respuestas.

El formato del final es de siete preguntas: cuatro de desarrollo (10 % o 15 % cada una), dos de opción múltiple con más de una respuesta posible (10 % y 15 %) y un ejercicio para graficar (25 %).

Parte 1. Modelo original de la cátedra

1. ¿Cuál es la forma clausal de Horn? ¿Por qué se aplica en el lenguaje Prolog? (10 %)
2. ¿Cuál es la diferencia entre algoritmo y proceso de búsqueda? (15 %)
3. ¿Qué son las reglas de inferencia? (10 %)
4. ¿Qué significa que una función sea destructiva? Compare con no destructiva y dé dos ejemplos. (15 %)
5. Elija las respuestas correctas (marque con x). El LISP fue en sus orígenes: (10 %)
    a. un programa que se usaba para evitar una explosión combinatoria
    b. un lenguaje compilado
    c. un lenguaje solamente recursivo
6. Elija las respuestas correctas (marque con x). El teorema de la confluencia hace significar que: (15 %)
    a. que admite forma normal
    b. que si dos variables que dependen de la definición de una función no pueden llegar a distintos resultados
    c. que es una expresión reducible
7. Grafique el árbol SLD del siguiente programa, para la consulta `hijos(W, julio).` (25 %)

```
varon(julio).
varon(julian).
varon(cesar).
mujer(teresita).
mujer(agustina).
padre(julio, julian).
padre(julio, cesar).
padre(julio, agustina).
madre(teresita, julian).
madre(teresita, cesar).
madre(teresita, agustina).
hijo(X, Y)  :- padre(Y, X), varon(X).
hija(X, Y)  :- padre(Y, X), mujer(X).
hijo(X, Y)  :- madre(Y, X), varon(X).
hija(X, Y)  :- madre(Y, X), mujer(X).
hijos(X, Y) :- hijo(X, Y).
hijos(X, Y) :- hija(X, Y).
```

Parte 2. Modelos nuevos basados en las diapositivas 2026

Modelo A

1. ¿Qué es un paradigma de programación? ¿Cuál es la diferencia entre el paradigma imperativo y el declarativo? (10 %)
2. ¿Cuál es la diferencia entre concurrencia y paralelismo? (15 %)
3. ¿Qué son las condiciones de Bernstein? (10 %)
4. ¿Qué diferencia hay entre una variable de ámbito léxico y una de ámbito dinámico? Dé un ejemplo de cada una. (15 %)
5. Elija las respuestas correctas (marque con x). Una lista punteada es: (10 %)
    a. una lista cuyo último CDR apunta a NIL
    b. una lista en la que el CDR de una estructura CONS apunta a un átomo
    c. una lista que ocupa una casilla CONS más que la lista común equivalente
6. Elija las respuestas correctas (marque con x). El predicado EQUAL devuelve T cuando: (15 %)
    a. los dos argumentos están referenciados por el mismo puntero
    b. los dos argumentos son estructuralmente iguales, es decir, sus valores imprimibles son iguales
    c. los argumentos son un entero y un real que representan el mismo valor
7. Dado el siguiente conjunto de sentencias, determine con las condiciones de Bernstein qué pares pueden ejecutarse concurrentemente, grafique el grafo de precedencia y escriba el programa usando cobegin/coend. (25 %)

```
S1 → a := x + y;
S2 → b := x * 2;
S3 → c := a + b;
S4 → d := b - z;
S5 → e := c + d;
```

Modelo B

1. ¿Qué es una estructura CONS? ¿Cuál es su relación con las listas en LISP? (10 %)
2. ¿Qué es la recursividad? Enumere sus características y escriba una función recursiva indicando el caso base y el caso recursivo. (15 %)
3. ¿Qué es el Lisp Listener? (10 %)
4. Compare las funciones CONS, LIST y APPEND. Indique qué devuelve cada una aplicada a los argumentos `'(a b)` y `'(c d)`. (15 %)
5. Elija las respuestas correctas (marque con x). Según la taxonomía de Flynn, la arquitectura en la que un solo flujo de instrucciones se aplica a múltiples conjuntos de datos es: (10 %)

   a. MISD

   b. SIMD

   c. MIMD
6. Elija las respuestas correctas (marque con x). En la forma COND: (15 %)

   a. se evalúan las formas de todas las cláusulas cuyo test sea NO-NIL

   b. se evalúan las formas de la primera cláusula cuyo test sea NO-NIL y se sale del COND

   c. si ningún test se cumple, devuelve NIL
7. Grafique con estructuras CONS (cajas CAR/CDR) las siguientes listas e indique cuántas casillas CONS utiliza cada una. (25 %)

```
(A (B C) D)
((A . B) C)
(A B . C)
```

Modelo C

1. Defina programación concurrente. ¿Qué significa que sea no determinista? (10 %)
2. Explique la metodología PCAM para el diseño de aplicaciones paralelas. (15 %)
3. ¿Qué es un predicado? Dé tres ejemplos. (10 %)
4. Explique la diferencia entre EQ, EQL, EQUAL y EQUALP. Dé un ejemplo en el que no todos devuelvan lo mismo. (15 %)
5. Elija las respuestas correctas (marque con x). La función SETQ: (10 %)
    a. evalúa todos sus argumentos
    b. requiere un número par de argumentos
    c. devuelve el resultado de la última evaluación
6. Elija las respuestas correctas (marque con x). En un sistema MIMD con memoria distribuida: (15 %)
    a. existe un único espacio de direcciones accesible a todos los procesadores
    b. los procesadores se comunican mediante el intercambio de mensajes
    c. cada procesador dispone de su propia memoria local
7. Dada la siguiente función, realice el seguimiento de la llamada `(cuenta '(3 a 5 b))` tal como lo mostraría TRACE, indicando cada entrada con sus argumentos, cada salida con su valor y el resultado final. (25 %)

```
(defun cuenta (l)
  (cond ((null l) 0)
        ((numberp (car l)) (+ 1 (cuenta (cdr l))))
        (t (cuenta (cdr l)))))
```

Parte 3. Banco de preguntas

Unidad I. Introducción: paradigmas y lenguajes

1. Defina paradigma y paradigma de programación.
2. Enumere los tipos de paradigmas de programación y nombre un lenguaje asociado a cada uno.
3. ¿Qué fórmula resume al paradigma imperativo y cuál al paradigma orientado a objetos?
4. ¿Qué es el paradigma declarativo y cuáles son sus dos modelos de desarrollo?
5. Explique la diferencia entre el paradigma imperativo y el declarativo.
6. Defina el paradigma funcional.
7. Defina el paradigma lógico. ¿Qué son los hechos, las reglas y las preguntas?
8. ¿Qué es el paradigma demostrativo?
9. Defina lenguaje de programación.
10. Clasifique los lenguajes de programación según su nivel de abstracción, su propósito, su forma de ejecución y su tipado.
11. ¿Qué son los meta-programas? Enumere las cinco estrategias para ejecutar un programa.
12. Compare la compilación con la interpretación.
13. Describa las cinco generaciones de lenguajes y dé un ejemplo de cada una.
14. ¿Qué es un estilo de programación?
15. ¿Qué establece el teorema de Dijkstra?
16. ¿De dónde proviene la programación funcional? Diferencie los lenguajes funcionales puros de los híbridos y dé ejemplos.
17. ¿En qué tipo de aplicaciones encuentra su hábitat natural la programación lógica?
18. ¿Qué tres elementos combina un objeto en la programación orientada a objetos? Indique dos diferencias entre la programación estructurada y la orientada a objetos.
19. ¿Qué es la programación orientada a aspectos? ¿Qué es una obligación transversal? Indique dos ventajas y dos desventajas.
20. Explique cómo funciona un programa dirigido por eventos.
21. ¿En qué consiste la programación con restricciones? ¿De qué está formado el modelo?
22. ¿Cuál es el objetivo de la programación por capas y cuál es su ventaja principal?
23. Indique verdadero o falso.
    a. En el paradigma imperativo se describe el problema y en el declarativo se describen paso a paso las instrucciones para resolverlo.
    b. Los lenguajes de tercera generación usan códigos independientes de la máquina.
    c. En Prolog el orden de ejecución de las instrucciones coincide con el orden en que fueron escritas.
    d. La programación con restricciones pertenece al paradigma declarativo.

e. En la interpretación hay una diferencia clara entre el tiempo de compilación y el tiempo de ejecución.

f. Lisp y Python se consideran lenguajes multiparadigma.

Unidad II. Programación funcional

24. ¿Quién propuso LISP y cuándo? ¿Qué significa el acrónimo? ¿Cómo era el lenguaje en sus orígenes?
25. ¿Qué significa que en LISP los programas y los datos son equivalentes?
26. Enumere y explique las características de Common Lisp.
27. ¿Cuáles son los tipos de valores de LISP? ¿Qué significa que el tipado sea dinámico?
28. ¿Qué es un átomo? Clasifique los átomos.
29. ¿Qué representan las constantes T y NIL?
30. Defina lista. Defínala recursivamente a partir de la estructura CONS.
31. ¿Qué es una lista punteada? Indique su ventaja y sus desventajas.
32. ¿Qué es el Lisp Listener y cómo se expresa su ciclo?
33. Enuncie la regla de evaluación de una expresión en LISP.
34. ¿Para qué sirve la función QUOTE?
35. ¿Qué devuelven las funciones `+`, `-`, `*` y `/` cuando no reciben argumentos?
36. ¿Qué diferencia hay entre las funciones aritméticas básicas y las funciones INCF y DECF?
37. ¿Qué es un predicado? ¿Cuáles de los predicados numéricos dan error si el argumento no es un número?
38. Enuncie las reglas de las operaciones aritméticas cuando algún término es una lista. ¿Qué devuelven `(+ 1 '(2 -3 4) 5)` y `(+ '(1 2 3) '(4 5))` en el intérprete de la cátedra?
39. ¿Qué diferencia hay entre una expresión simbólica y una forma?
40. ¿Qué devuelve cada una de las siguientes expresiones?
    a. `(car '((a b) c))`
    b. `(cdr '((a b c)))`
    c. `(cadr '(a b c))`
    d. `(last '(a b c))`
    e. `(cdr '(a . b))`
41. Explique la sintaxis y las reglas de la función SETQ.
42. Compare las funciones CONS, LIST y APPEND.
43. ¿Qué significa que una función sea destructiva? Compare con no destructiva y dé dos ejemplos de cada una.
44. ¿Qué devuelve la función MEMBER? ¿Por qué `(member '(x y) '(a (x y) d))` devuelve NIL?
45. ¿Qué devuelven ATOM, CONSP, LISTP y NULL cuando se aplican a NIL?
46. Enumere los niveles de igualdad de LISP y el predicado que corresponde a cada uno.
47. ¿Qué devuelve cada una de las siguientes expresiones?

a. `(eq 1 1.0)`

b. `(eql 1.0 1.0)`

c. `(equal "a" "a")`

d. `(eql '(a b) '(a b))`

e. `(equal '(a b) '(a b))`

f. `(equalp "a" "A")`

48. ¿Cómo evalúan sus argumentos AND y OR? ¿Qué devuelven `(and t (< 2 5) (* 2 5))` y `(or nil (= 4 5) (rem 23 13))`?

49. ¿Qué diferencia hay entre PROGN, PROG1 y PROG2?

50. Enumere las estructuras condicionales de LISP. ¿En qué se diferencian WHEN y UNLESS?

51. Explique el funcionamiento de COND. ¿Qué devuelve si ningún test se cumple?

52. Enumere las estructuras iterativas de LISP. ¿Cuáles son las tres partes de la estructura DO?

53. ¿Qué diferencia hay entre MAPCAR y MAPLIST? ¿Qué devuelven `(mapcar '+ '(7 8 9) '(1 2 3))` y `(maplist '+ '(7 8 9) '(1 2 3))`?

54. ¿Qué diferencia hay entre PRINT, PRIN1 y PRINC?

55. En la función FORMAT, ¿qué diferencia hay entre usar `t` o `nil` como destino? Nombre tres directivas.

56. ¿Qué diferencia hay entre SETF y SETQ?

57. ¿Para qué sirve LET? ¿Qué diferencia hay entre LET y LET*?

58. ¿Qué es una lambda-expresión? Dé un ejemplo.

59. ¿Cuáles son las partes de una definición con DEFUN? ¿Qué devuelve la evaluación de un DEFUN y qué devuelve la llamada a la función?

60. Enuncie las reglas de la ligadura de parámetros en una función.

61. ¿Qué diferencia hay entre el ámbito léxico y el ámbito dinámico? ¿Para qué sirven DEFVAR, DEFPARAMETER y DEFCONSTANT?

62. Defina recursividad y enumere las características de la programación recursiva.

63. ¿Para qué sirve TRACE? ¿Cómo se desactiva?

64. ¿Qué es un stream? ¿Para qué sirven OPEN, CLOSE y LOAD?

65. ¿Cuál es la convención para los comentarios en LISP?

66. Indique verdadero o falso.

    a. El ciclo del Lisp Listener es (READ (EVAL (PRINT))).

    b. Las operaciones aritméticas modifican el valor de sus argumentos.

    c. En las operaciones matemáticas, si al menos un término es una lista, el resultado es una lista con la estructura de la lista de menor profundidad.

    d. Una variable de ámbito léxico se asocia a las variables globales.

    e. La función LIST puede crear listas impropias.

    f. La recursividad es una estructura de datos.

    g. LET asigna los valores a sus variables en paralelo.

    h. MEMBER compara por defecto con EQL.

Unidad III. Programación concurrente y paralela

67. Enumere las características de los programas secuenciales. ¿Qué significa que sean deterministas?
68. Defina concurrencia.
69. ¿En qué consiste el no determinismo de los programas concurrentes, con un único procesador y con varios?
70. ¿Qué acontecimientos impulsaron la programación concurrente?
71. ¿Por qué es necesaria la programación concurrente?
72. Compare concurrencia y paralelismo.
73. ¿Qué diferencia hay entre paralelo y distribuido? Defina sistema distribuido de computación.
74. ¿Qué es la computación paralela?
75. ¿De qué tres formas puede ejecutarse un programa concurrente?
76. Enumere las características de los programas concurrentes. ¿Qué se requiere para validarlos?
77. Enumere las ventajas de la programación concurrente.
78. Explique la taxonomía de Flynn.
79. ¿Qué es SPMD?
80. Compare los sistemas MIMD con memoria compartida y con memoria distribuida.
81. ¿Qué diferencia hay entre UMA y NUMA?
82. ¿Qué es un sistema MPP?
83. Enumere las características deseadas de una aplicación paralela y defina dos de ellas.
84. ¿Respecto de qué tres aspectos puede haber transparencia? ¿Qué diferencia hay entre portabilidad de código y portabilidad de performance?
85. Explique las tres estrategias de descomposición de problemas.
86. Explique las cuatro etapas de la metodología PCAM. ¿En qué se enfocan las dos primeras y en qué las dos últimas?
87. ¿En qué consiste el mapeo de procesos y por qué es complicado decidirlo?
88. ¿Qué es el scaling down?
89. Enuncie las condiciones de Bernstein. ¿Qué son los conjuntos de lectura y de escritura?
90. ¿Qué es un grafo de precedencia?
91. ¿Cómo se especifica la concurrencia con el par cobegin/coend?
92. Aplique las condiciones de Bernstein al siguiente conjunto de sentencias, grafique el grafo de precedencia y escriba el programa con cobegin/coend.

```
S1 → p := a + b;
S2 → q := a * c;
S3 → r := p + q;
S4 → s := r * 2;
S5 → t := q - 1;
```

93. Indique verdadero o falso.

a. Los programas concurrentes son deterministas.

b. La concurrencia implica necesariamente la existencia de múltiples procesadores.

c. Dos sentencias que leen la misma variable no pueden ejecutarse concurrentemente.

d. En la práctica no se han construido máquinas MISD.

e. En un sistema UMA todos los procesadores tienen el mismo tiempo de acceso a memoria.

f. Un grafo de precedencia puede tener ciclos.

Temas del programa que no están en las diapositivas 2026

94. ¿Qué es una cláusula de Horn y qué tipos hay?

95. ¿Qué son la unificación y el backtracking en Prolog?

96. ¿Qué es un árbol SLD?

97. En el cálculo lambda, ¿qué son un redex, una reducción beta y una forma normal?

98. ¿Qué establece el teorema de confluencia (Church-Rosser)?

99. ¿Qué son la exclusión mutua y la sección crítica? ¿Qué es un semáforo?

100. ¿Qué es un interbloqueo (deadlock)?

Parte 4. Respuestas

Las respuestas de las Unidades I, II y III salen de las diapositivas 2026 y remiten al apunte correspondiente de la carpeta `apuntes/`. Las respuestas sobre lógica, Prolog, cálculo lambda y sincronización (preguntas 1, 2, 3, 6 y 7 del modelo original y 94 a 100 del banco) no están desarrolladas en las diapositivas 2026: son respuestas de teoría general y conviene contrastarlas con el material de la cátedra de ese tema.

Modelo original

1. Una cláusula es una disyunción de literales. Una cláusula de Horn es la que tiene como máximo un literal positivo. Escrita como implicación queda `H :- B1, B2, ..., Bn`, que se lee "H es verdadero si B1 y B2 y ... y Bn lo son". Según cuántos literales tenga hay tres casos: hecho (solo el literal positivo, sin cuerpo), regla (un literal positivo y uno o más negativos) y objetivo o consulta (sin literal positivo). Se aplica en Prolog porque restringe la lógica de primer orden a un subconjunto en el que la demostración se puede mecanizar de forma eficiente con una sola regla de inferencia, la resolución: cada cláusula admite una lectura procedimental ("para demostrar H, demostrar B1, ..., Bn"), lo que permite ejecutarla como un programa. En las diapositivas (apunte 01): el paradigma lógico se basa en un subconjunto del cálculo de predicados, con instrucciones escritas como cláusulas de Horn, y permite un método mecánico de demostración llamado resolución; las instrucciones de Prolog se llaman reglas o cláusulas de Horn.

2. Un algoritmo es una secuencia finita y ordenada de pasos, definida por quien programa, que indica cómo llegar a la solución; el control lo fija el programador y, ante la misma entrada, sigue siempre el mismo camino. Un proceso de búsqueda es la exploración de un espacio de alternativas (un árbol de búsqueda) aplicando reglas hasta encontrar una o más soluciones; el camino no está escrito de antemano, sino que lo decide una estrategia (en Prolog, profundidad con vuelta atrás), puede haber ramas que fallan, varias soluciones o incluso ramas que no terminan. En programación lógica el programador aporta la lógica (hechos y reglas) y el intérprete aporta el control mediante la búsqueda; es la idea de Kowalski: Algoritmo = Lógica + Control.

3. Son esquemas que permiten obtener una fórmula nueva (la conclusión) a partir de otras ya aceptadas (las premisas) de manera puramente sintáctica, garantizando que si las premisas son verdaderas la conclusión también lo es. Ejemplos: modus ponens (de A y A → B se obtiene B), modus tollens y la resolución. Prolog usa una sola regla de inferencia, la resolución, combinada con la unificación. En las diapositivas (apunte 01): el paradigma lógico puede deducir nuevos hechos a partir de otros hechos conocidos mediante mecanismos internos de inferencia.

4. Una función destructiva es la que modifica el contenido de alguno de sus parámetros. Una no destructiva devuelve un resultado sin modificar sus argumentos, por lo que para conservar el resultado hay que asignarlo. Ejemplo 1: con `(setq a '(1 2) b '(3 4))`, `(append a b)` devuelve `(1 2 3 4)` y `a` sigue valiendo `(1 2)`; en cambio `(nconc a b)` devuelve `(1 2 3 4)` y `a` pasa a valer `(1 2 3 4)`. Ejemplo 2: `(cons 9 b)` devuelve `(9 3 4)` sin cambiar `b`; `(push 9 b)` deja `b` en `(9 3 4)`. Otras destructivas: RPLACA, RPLACD, POP, INCF, DECF. Otras no destructivas: REVERSE, BUTLAST, las aritméticas. (Apunte 03.)

5. La respuesta es la c. Según las diapositivas, LISP fue originalmente un lenguaje simple y pequeño que implementa el modelo de las funciones recursivas, y sus principales fundamentos de control son la recursión y las condicionales simples. No es la b porque en su origen es un lenguaje interpretado (el ciclo READ-EVAL-PRINT). La opción a no corresponde a ninguna definición de la teoría. (Apunte 02.)

6. La respuesta es la b. El teorema de confluencia (Church-Rosser) dice que si una expresión puede reducirse por dos caminos distintos, siempre existe una expresión común a la que ambos resultados pueden seguir reduciéndose; por lo tanto el resultado no depende del orden de reducción y una expresión no puede llegar a dos resultados (formas normales) distintos. No es la a: la confluencia garantiza que la forma normal, si existe, es única, pero no garantiza que exista. No es la c: "expresión reducible" es la definición de redex.

7. La consulta tiene tres soluciones, en este orden: `W = julian`, `W = cesar`, `W = agustina`. El árbol se construye resolviendo siempre el objetivo de más a la izquierda y probando las cláusulas en el orden en que están escritas.

```
hijos(W, julio)
├─ [hijos, cláusula 1]  hijo(W, julio)
│   ├─ [hijo, cláusula 1]  padre(julio, W), varon(W)
│   │   ├─ W = julian    →  varon(julian)    →  éxito:  W = julian
│   │   ├─ W = cesar     →  varon(cesar)     →  éxito:  W = cesar
│   │   └─ W = agustina  →  varon(agustina)  →  fallo
│   └─ [hijo, cláusula 2]  madre(julio, W), varon(W)
│       └─ no hay ningún hecho madre(julio, _)   →  fallo
└─ [hijos, cláusula 2]  hija(W, julio)
    ├─ [hija, cláusula 1]  padre(julio, W), mujer(W)
    │   ├─ W = julian    →  mujer(julian)    →  fallo
    │   ├─ W = cesar     →  mujer(cesar)     →  fallo
    │   └─ W = agustina  →  mujer(agustina)  →  éxito:  W = agustina
    └─ [hija, cláusula 2]  madre(julio, W), mujer(W)
        └─ no hay ningún hecho madre(julio, _)   →  fallo
```

Modelo A

1. Un paradigma de programación es un patrón de pensamiento para la resolución de problemas; una forma de representar y manipular el conocimiento, que representa un enfoque particular o filosofía para la construcción del software. En el paradigma imperativo se describe paso a paso el conjunto de instrucciones que deben ejecutarse para variar el estado del programa y hallar la solución (un algoritmo: el cómo). En el declarativo las sentencias describen el problema que se quiere solucionar (el qué), pero no las instrucciones para resolverlo; eso lo hacen mecanismos internos de inferencia. (Apunte 01.)

2. En el paralelismo hay procesamiento simultáneo físicamente: implica múltiples elementos de procesamiento con funcionamiento independiente. En la concurrencia el cómputo es simultáneo lógicamente: no implica necesariamente varios procesadores, porque puede ser simulada por el sistema operativo sobre una CPU. La concurrencia es una característica de las tareas (la capacidad de poder realizarse en paralelo); el paralelismo se asocia a los recursos disponibles para realizarlas a la vez. Ejemplo: un programa multihilo es paralelo en una máquina multiprocesador y concurrente en un solo procesador. (Apunte 06.)

3. Son las condiciones que permiten determinar si dos conjuntos de instrucciones Si y Sj se pueden ejecutar concurrentemente. Se definen L(Sk), el conjunto de variables que se leen, y E(Sk), el conjunto de variables que se escriben. Deben cumplirse las tres: $L(Si) \cap E(Sj) = \emptyset$, $E(Si) \cap L(Sj) = \emptyset$ y $E(Si) \cap E(Sj) = \emptyset$. (Apunte 07.)

4. Las variables son de ámbito léxico a menos que se declaren de otra forma: solo pueden verse dentro del texto de la función o forma donde se definen. Ejemplo: el parámetro `x` de `(defun cuadrado (x) (* x x))`, o las variables de un LET; fuera de ahí dan error de variable no ligada. Las variables declaradas especiales son de ámbito dinámico: son variables libres accesibles desde cualquier parte del programa a partir del punto en que se definieron (la idea de variable global). Ejemplo: `(defvar contador 0)`; también DEFPARAMETER y DEFCONSTANT. (Apunte 05.)

5. La respuesta es la b. La a describe una lista común (propia). La c es falsa: la lista punteada ahorra una casilla CONS. (Apunte 02.)

6. Las respuestas son la a y la b. Si dos argumentos son el mismo objeto (EQ), también son EQUAL. La c es falsa: `(equal 5 5.0)` devuelve NIL; para eso hacen falta `=` o EQUALP. (Apunte 03.)

7. Conjuntos de lectura y escritura:

Sentencia	L	E
S1	{x, y}	{a}
S2	{x}	{b}
S3	{a, b}	{c}
S4	{b, z}	{d}
S5	{c, d}	{e}

Pares que no cumplen: S1-S3 ($E(S1) \cap L(S3) = \{a\}$), S2-S3 ({b}), S2-S4 ({b}), S3-S5 ({c}), S4-S5 ({d}). Todos los demás cumplen las tres condiciones. S1 y S2 leen ambas `x`, pero eso no impide la concurrencia.

	S1	S2	S3	S4	S5
S1	-	Sí	No	Sí	Sí
S2	-	-	No	No	Sí
S3	-	-	-	Sí	No
S4	-	-	-	-	No




```
begin
  cobegin
     a := x + y;
     b := x * 2;
  coend
  cobegin
     c := a + b;
     d := b - z;
  coend
  e := c + d;
end
```

Con cobegin/coend S4 queda esperando también a S1, aunque en el grafo solo depende de S2; es una restricción de más que no cambia el resultado.

Modelo B

1. Un CONS es una estructura de información que contiene dos componentes, el CAR y el CDR. Se usa fundamentalmente para representar listas: una lista se define recursivamente como la lista vacía o como un CONS cuyo CDR es una lista. Una lista es entonces una cadena de CONSES enlazados por sus CDR y terminada en NIL. El CAR de cada CONS apunta a un elemento y el CDR al siguiente CONS (o a NIL); hay un CONS por cada elemento. Un átomo y la lista vacía no son estructuras CONS. (Apunte 02.)

2. Un proceso es recursivo cuando está basado en su propia definición. No es una estructura de datos sino una técnica de programación que permite que un bloque de instrucciones se ejecute n veces, y muchas veces reemplaza a las estructuras repetitivas. Características: implementación intuitiva y elegante; traducción casi inmediata de la solución (caso base y caso recursivo) a código; útil cuando hay varios niveles de anidamiento; la comprensión del código puede ser compleja. Ejemplo: `(defun factorial (n) (if (= n 0) 1 (* n (factorial (- n 1)))))`; el caso base es `n = 0`, que devuelve 1, y el caso recursivo es `(* n (factorial (- n 1)))`. (Apunte 05.)

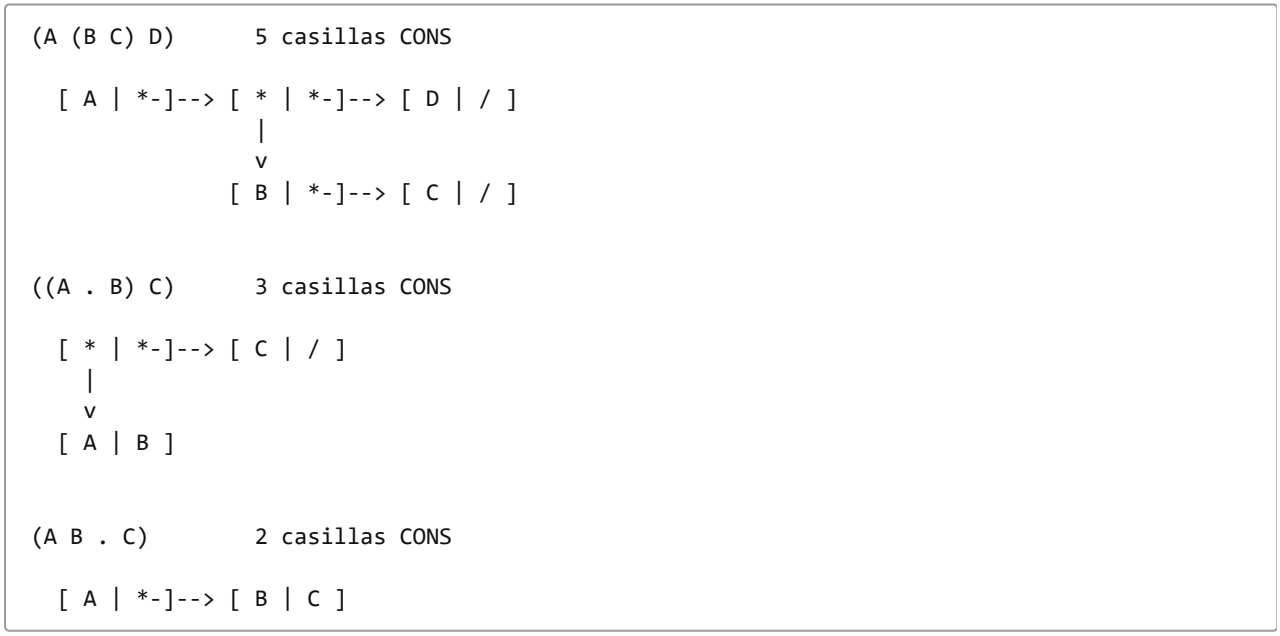
3. Es el ciclo evaluador que LISP realiza continuamente: espera una entrada, la lee, la evalúa, imprime el resultado y vuelve a empezar. Se lo llama ciclo READ-EVAL-PRINT y se expresa como `(PRINT (EVAL (READ)))`. (Apunte 02.)

4. Las tres construyen listas. CONS recibe dos argumentos y crea una nueva estructura cons cuyo CAR es el primero y cuyo CDR es el segundo. LIST recibe cualquier cantidad de argumentos y devuelve una lista con todos ellos como elementos; solo crea listas propias. APPEND concatena listas; todos sus argumentos deben ser listas salvo el último. Resultados: `(cons '(a b) '(c d))` devuelve `((a b) c d)`, de 3 elementos; `(list '(a b) '(c d))` devuelve `((a b) (c d))`, de 2 elementos; `(append '(a b) '(c d))` devuelve `(a b c d)`, de 4 elementos. (Apunte 03.)

5. La respuesta es la b, SIMD (Single Instruction, Multiple Data). (Apunte 06.)

6. Las respuestas son la b y la c. (Apunte 04.)

7. Cada caja es `[CAR | CDR]` y `/` indica CDR = NIL.



Modelo C

1. Es el conjunto de anotaciones y técnicas utilizadas para describir mediante programas el paralelismo potencial de los problemas, así como para resolver los problemas de comunicación y sincronización que se presentan cuando varios procesos que se ejecutan concurrentemente comparten recursos. Es no determinista porque, ante un instante de tiempo, se desconoce qué va a ocurrir en el siguiente: con un único procesador no se sabe si después de una instrucción habrá una interrupción que le dé el procesador a otro proceso; con varios procesadores, sus velocidades no están sincronizadas y no se sabe cuál ejecutará primero su siguiente instrucción. (Apunte 06.)

2. Estructura el diseño en cuatro etapas. Partición: el cómputo y los datos se descomponen en pequeñas tareas, buscando oportunidades de ejecución concurrente (descomposición del dominio o funcional). Comunicación: se determinan las comunicaciones necesarias para coordinar las tareas. Aglomeración: se evalúan las tareas y la comunicación respecto de prestaciones y costes, y si conviene se combinan en tareas mayores para reducir costes de comunicación. Mapping: cada tarea se asigna a un procesador, buscando maximizar su utilización y minimizar la comunicación; puede ser estático o por balanceo de carga en ejecución. Las dos primeras etapas se enfocan en la concurrencia y la escalabilidad; las dos últimas, en la localidad y las prestaciones. (Apunte 07.)

3. Un predicado es una función booleana: devuelve solo los valores T o NIL. Ejemplos: `(numberp 7)` devuelve T; `(evenp 8)` devuelve T; `(null '(a))` devuelve NIL. (Apuntes 02 y 03.)

4. Son cuatro niveles de igualdad, cada uno más permisivo que el anterior. EQ: identidad referencial, verdadero si los dos argumentos están referenciados por el mismo puntero. EQL: identidad representacional, verdadero para el mismo objeto o para números del mismo tipo y valor. EQUAL: igualdad estructural, verdadero si los valores imprimibles son iguales; sirve para listas y strings. EQUALP: igualdad de valores sin distinguir mayúsculas ni tipo numérico. Ejemplo: con `'(a b)` y `'(a b)`, EQ y EQL devuelven NIL, y EQUAL y EQUALP devuelven T. Otro: con `1` y `1.0`, solo EQUALP devuelve T. (Apunte 03.)

5. Las respuestas son la b y la c. La a es falsa: los argumentos impares (los nombres de variable) no se evalúan; de ahí la Q de quote. (Apunte 03.)

6. Las respuestas son la b y la c. La a describe a los sistemas de memoria compartida. (Apunte 06.)

7. La función cuenta cuántos elementos numéricos hay en la lista. El resultado es 2.

```
Entering: CUENTA, Argument list: ((3 A 5 B))
 Entering: CUENTA, Argument list: ((A 5 B))
  Entering: CUENTA, Argument list: ((5 B))
   Entering: CUENTA, Argument list: ((B))
    Entering: CUENTA, Argument list: (NIL)
    Exiting: CUENTA, Value: 0
   Exiting: CUENTA, Value: 0
  Exiting: CUENTA, Value: 1
 Exiting: CUENTA, Value: 1
Exiting: CUENTA, Value: 2
2
```

Banco, Unidad I

1. Un paradigma está constituido por los supuestos teóricos generales, las leyes y las técnicas para su aplicación que adoptan los miembros de una determinada comunidad científica; es un marco de referencia que impone reglas e indica qué es válido y qué queda fuera. Un paradigma de programación es un patrón de pensamiento para la resolución de problemas: una forma de representar y manipular el conocimiento, un enfoque o filosofía para construir software. Ninguno es mejor que otro.

2. Imperativo (C, BASIC, Pascal); procedimental (extensión del imperativo); declarativo, con sus modelos funcional (LISP, Scheme) y lógico (Prolog); demostrativo (programación por ejemplos, modelo genético); orientado a objetos (Smalltalk); reactivo (RxJS, React, Akka Streams); basado en componentes (React, Angular, Vue); concurrente (Java, Python) y paralelo (Go, CUDA, OpenMP, MPI).

3. Imperativo: Algoritmos + Estructura de Datos = Programa. Orientado a objetos: Objetos + Mensajes = Programa.

4. Es el paradigma en el que los programas se desarrollan especificando o declarando un conjunto de condiciones, proposiciones, afirmaciones, restricciones, ecuaciones o transformaciones que describen el problema y detallan su solución; la solución se obtiene mediante mecanismos internos de control, sin especificar exactamente cómo encontrarla. Sus modelos son el funcional y el lógico.

5. En el imperativo se describe paso a paso un conjunto de instrucciones que varían el estado del programa (un algoritmo: el cómo). En el declarativo las sentencias describen el problema (el qué) y la resolución queda a cargo de mecanismos internos de inferencia.

6. Es un modelo matemático de composición funcional en el que el resultado de un cálculo es la entrada del siguiente, hasta que una composición produce el valor deseado. Opera solamente a través de funciones; cada función devuelve un solo valor dada una lista de parámetros. Permite que un programa se modifique a sí mismo.

7. Se basa en un subconjunto del cálculo de predicados, con instrucciones escritas como cláusulas de Horn. Representa conocimiento mediante relaciones (predicados) entre objetos (datos) y deduce nuevos hechos a partir de los conocidos mediante resolución. Los hechos y las reglas especifican las relaciones; las preguntas son las consultas cuya demostración constituye la ejecución del programa.

8. También llamado programación por ejemplos: el programador no especifica procedimentalmente cómo construir una solución, sino que presenta soluciones de problemas similares. Su modelo de desarrollo es el genético.

9. Es un conjunto de sintaxis y reglas semánticas que definen los programas del computador; una técnica estándar de comunicación para entregarle instrucciones. Para ejecutarse, el programa debe traducirse a lenguaje de máquina mediante compilación, interpretación o un proceso intermedio.

10. Por nivel de abstracción: bajo nivel (ensamblador) y alto nivel (Python, Java, C#). Por propósito: general (C++, Python, Java) y de dominio específico (SQL, HTML). Por forma de ejecución: compilados, interpretados e híbridos. Por tipado: estático, verificado en compilación (C++, Java), y dinámico, en ejecución (Python, JavaScript).

11. Son programas (compiladores, intérpretes, etc.) cuyos datos de entrada son el código fuente de otros programas. Las estrategias son: compilación, interpretación, ejecución en máquina virtual, enlazado de rutinas y librerías, y preprocesamiento.

12. En la compilación hay dos momentos distintos, tiempo de compilación y tiempo de ejecución, y se gana eficiencia (C, C++). En la interpretación no hay diferencia entre ambos tiempos y se gana flexibilidad: el código se puede construir y ejecutar sobre la marcha (BASIC, LISP, Scheme, Python, Ruby).

13. Primera: código binario, lenguaje de máquina propio de cada ordenador. Segunda: lenguajes simbólicos o ensambladores (add, move). Tercera: lenguajes de alto nivel, con códigos independientes de la máquina (C, Java, Python). Cuarta: herramientas para construir aplicaciones combinando piezas prefabricadas (SQL, MATLAB). Quinta: programación lógica y sistemas expertos, asociada a la inteligencia artificial (Prolog, LISP).

14. Son las convenciones utilizadas para escribir código fuente en los distintos lenguajes; también se llaman estándares o guías. Ejemplos: nomenclatura, indentación, comentarios, estructuras.

15. Que todo programa puede escribirse utilizando únicamente tres instrucciones de control: secuencia, instrucción condicional e iteración. De ahí surge la programación estructurada, que no usa GOTO.

16. Del cálculo lambda, teoría matemática elaborada por Alonzo Church. Los puros no admiten conceptos imperativos (Haskell, Miranda); los híbridos son menos dogmáticos y admiten secuencias de instrucciones y asignación de variables (Lisp, Scheme, Ocaml, Standard ML).

17. En aplicaciones de inteligencia artificial: sistemas expertos, demostración automática de teoremas y reconocimiento de lenguaje natural.

18. Estado (datos), comportamiento (métodos) e identidad. Diferencias: en la estructurada se escriben funciones y después se les pasan datos, mientras que en la orientada a objetos se definen objetos con datos y métodos y se les envían mensajes; la orientada a objetos une los tipos abstractos de datos con sus operaciones (clase) e incorpora polimorfismo y envío de mensajes.

19. Es un paradigma que trata las obligaciones transversales de los programas como módulos separados (aspectos) para lograr una correcta separación de responsabilidades. Una obligación transversal es la que se repite en varias partes de un programa sin que esas partes tengan relación directa. Ventajas: modulariza programas extensos, vuelve más limpio el código, puede mezclarse con cualquier otro paradigma. Desventajas: sufre el antipatrón de acciones a distancia, vuelve difícil de comprender el código, es complicado saber cuándo conviene usarla.

20. Al comenzar hace las inicializaciones y queda bloqueado hasta que se produzca algún evento. Cuando ocurre uno de los eventos esperados, ejecuta el código del manejador (administrador) de ese evento. La estructura y la ejecución quedan determinadas por los sucesos que ocurren en el sistema.

21. Las relaciones entre las variables se expresan como restricciones (ecuaciones) que cualquier solución debe satisfacer, en lugar de indicar los pasos para obtenerla. El modelo está formado por variables, con sus posibles valores, y restricciones entre ellas; el sistema encuentra las soluciones. Es declarativa y se usa en problemas combinatorios como la planificación.

22. Separar la lógica de negocios de la lógica de diseño (por ejemplo, la capa de datos de la de presentación). Su ventaja principal es que el desarrollo se hace en varios niveles y, ante un cambio, solo se modifica el nivel afectado; cada grupo solo necesita conocer la API entre niveles.

23. a. Falso: es al revés. b. Verdadero. c. Falso: no tiene nada que ver con el orden en que fueron escritas. d. Verdadero. e. Falso: en la interpretación no hay diferencia entre ambos tiempos. f. Verdadero.

Banco, Unidad II

24. John McCarthy, a finales de los años 50. LISt Processing. Era un lenguaje simple y pequeño, con funciones de construcción y acceso a listas, definición de nuevas funciones, pocos predicados de igualdad y funciones para evaluar llamadas; sus fundamentos de control son la recursión y las condicionales simples, y no posee asignaciones.

25. Que las listas son la base tanto de los programas como de los datos: el código tiene la misma estructura que los datos (principio de homoiconicidad). Una llamada a función es una lista, y una lista puede tratarse como dato usando quote.

26. Portabilidad: excluye lo que no pueda implementarse en la mayoría de las máquinas. Consistencia: intérprete y compilador asignan la misma semántica. Expresividad: recoge las construcciones más útiles de dialectos anteriores. Eficiencia: facilita la producción de código compilado de alta calidad. Potencia: paquetes que corren sobre el núcleo. Estabilidad: el núcleo cambia lentamente y por acuerdo de los expertos del estándar.

27. El tipo atom (número, carácter, símbolo, string) y el tipo cons (lista); además, estructuras y tipos definidos por el usuario. Es dinámico porque el tipo depende del momento en que se hace la asignación: no es necesario declarar el tipo del contenido.

28. Es un elemento indivisible con significado propio. Hay números (enteros, racionales, coma flotante, complejos), caracteres, strings y átomos simbólicos (nombres de variables y funciones), más las constantes especiales T y NIL.

29. T representa verdadero. NIL representa falso o la lista vacía. Ambas son símbolos reservados que apuntan a sí mismos.

30. Una lista es una estructura básica de datos y de programa, delimitada por paréntesis y formada por una secuencia de átomos y/o listas. Recursivamente: una lista es la lista vacía o un CONS cuyo CDR es una lista.

31. Es la que se forma cuando el CDR de un CONS apunta a un átomo en lugar de a otro CONS o a NIL; por ejemplo `(A . 4)`. Ventaja: ahorra una casilla CONS. Desventajas: se pierde flexibilidad porque no hay marca de fin de lista, y no admite agregarle nuevas casillas.

32. Es el ciclo evaluador que LISP realiza continuamente, siempre a la espera de entradas: lee, evalúa e imprime. Es el ciclo READ-EVAL-PRINT, `(PRINT (EVAL (READ)))`.

33. Si la expresión es un átomo numérico, el resultado es ese mismo átomo. Si es de la forma `(f a1 ... an)`, el intérprete evalúa recursivamente cada argumento y luego aplica `f` a los valores obtenidos.

34. Provoca que un objeto no sea evaluado. Es la forma de pasar una lista como dato y no como llamada a función. Se escribe `(quote x)` o `'x`.

35. + devuelve 0; * devuelve 1; - y / dan error.

36. Las aritméticas básicas no modifican el valor de sus argumentos: solo devuelven un resultado. INCF y DECF incrementan o decrementan una variable y modifican su valor: son destructivas.

37. Es una función booleana que devuelve solo T o NIL. NUMBERP e INTEGERP aceptan cualquier objeto. ODDP, EVENP y ZEROP dan error si el argumento no es del tipo esperado.

38. Los términos deben ser numéricos; si todos son átomos el resultado es un átomo; si al menos uno es una lista, el resultado es una lista con la estructura de la lista de mayor profundidad; si hay más de una lista, deben tener la misma longitud en el primer nivel; se opera de izquierda a derecha, término a término. `(+ 1 '(2 -3 4) 5)` devuelve `(8 3 10)`. `(+ '(1 2 3) '(4 5))` da el error "arguments not all the same length".

39. Una expresión simbólica es cualquier átomo válido o lista. Una forma es un objeto que puede ser evaluado.

40. a. `(A B)`. b. `NIL`. c. `B`. d. `(C)`. e. `B`.

41. `(SETQ var forma var forma ...)`. Asigna a cada variable el valor de su forma. Requiere un número par de argumentos; los impares (las variables) no se evalúan; las evaluaciones y asignaciones son secuenciales; devuelve el resultado de la última evaluación.

42. CONS: dos argumentos, crea un nuevo cons con el primero como CAR y el segundo como CDR; puede crear listas punteadas. LIST: cualquier cantidad de argumentos, devuelve una lista con todos como elementos; solo listas propias. APPEND: concatena listas; todos los argumentos deben ser listas salvo el último.

43. Ver la respuesta 4 del modelo original.

44. Devuelve la lista cuyo CAR es el elemento buscado y cuyo CDR es el resto de la lista, o NIL si no lo encuentra. Devuelve NIL en ese caso porque por defecto compara con EQL, y las dos listas `(x y)` son objetos distintos en memoria; con `:test #'equal` devuelve `((X Y) D)`.

45. ATOM devuelve T, CONSP devuelve NIL, LISTP devuelve T y NULL devuelve T.

46. Igualdad numérica: `=`. Identidad referencial: EQ. Identidad representacional: EQL. Igualdad estructural: EQUAL. Igualdad de valores: EQUALP.

47. a. NIL. b. T. c. T. d. NIL. e. T. f. T.

48. Evalúan sus argumentos en orden y no necesariamente todos. AND se detiene en el primer NIL y devuelve NIL; si todos son NO-NIL devuelve el valor del último. OR se detiene en el primer NO-NIL y devuelve ese valor; si todos son NIL devuelve NIL. Los dos ejemplos devuelven 10.

49. Las tres evalúan todas sus formas en secuencia. PROGN devuelve el valor de la última, PROG1 el de la primera y PROG2 el de la segunda.

50. IF, COND, WHEN, UNLESS, CASE y TYPECASE. WHEN evalúa sus formas cuando el test es verdadero; UNLESS las evalúa cuando el test es NIL.

51. Evalúa los tests de las cláusulas en orden hasta encontrar el primero que sea NO-NIL; evalúa las formas de esa cláusula, devuelve el valor de la última y sale, sin mirar las cláusulas siguientes. Si la cláusula no tiene formas devuelve el valor del test. Si ningún test se cumple devuelve NIL.

52. LOOP, DO, DOTIMES y DOLIST. Las partes del DO son la lista de parámetros (con valor inicial y paso), el test de final (con el resultado) y el cuerpo. Las variables se asignan en paralelo y el test se evalúa antes que el cuerpo.

53. MAPCAR aplica la función a los sucesivos CAR de las listas; MAPLIST la aplica a los sucesivos CDR. El primero devuelve `(8 10 12)` y el segundo `((8 10 12) (10 12) (12))`.

54. PRINT escribe el objeto en una nueva línea, tal como lo aceptaría READ. PRIN1 hace lo mismo sin salto de línea. PRINC tampoco salta de línea e imprime los strings sin comillas.

55. Con `t` imprime en la salida por defecto y devuelve NIL. Con `nil` no imprime: crea el string y lo devuelve. Directivas: `~A` (un objeto cualquiera), `~D` (número en base 10), `~S` (expresión simbólica), `~%` (nueva línea).

56. SETQ asigna a una variable. SETF asigna a un lugar, que puede ser una variable o una forma que designa una posición, como `(car x)`.

57. Crea variables locales, que solo existen dentro de su cuerpo; los valores previos de variables con el mismo nombre se guardan y se restauran al terminar. LET asigna los valores en paralelo; LET* los asigna secuencialmente, de modo que una variable puede usar a las anteriores.

58. Es una función sin nombre, con la misma estructura que una función. Ejemplo: `((lambda (a b) (* a b)) 4 8)` devuelve 32.

59. Nombre de la función, lista de argumentos (lambda-lista) y cuerpo; opcionalmente declaraciones y string de documentación. Evaluar el DEFUN devuelve el nombre de la función. La llamada devuelve el valor de la última forma evaluada en el cuerpo.

60. La lista de parámetros apunta a los mismos valores que los argumentos de la llamada; al terminar la función los parámetros se desligan; no afectan a las variables externas; no pueden ser accedidos por otras funciones llamadas desde ella, salvo que se pasen como argumentos.

61. Ver la respuesta 4 del modelo A. DEFVAR declara una variable global; DEFPARAMETER lo mismo, pero para usarla como parámetro; DEFCONSTANT declara una constante, cuyo valor no puede cambiarse.

62. Ver la respuesta 2 del modelo B.

63. Muestra el nombre de la función y sus argumentos al entrar, y el nombre y su valor al salir, sin cambiar el código fuente. Se desactiva con UNTRACE; sin argumentos detiene el seguimiento de todas las funciones.

64. Es un tipo de datos que mantiene la información sobre el dispositivo o fichero al que está asociado, y hace que la entrada/salida sea independiente del dispositivo. OPEN devuelve un stream sobre un fichero, con `:direction :input`, `:output` o `:io`; CLOSE lo cierra; LOAD carga un fichero en el intérprete.

65. Cuatro `;` al comienzo del fichero para indicar su contenido; tres para comentar una función, en la línea posterior a su cabecera; dos para comentar una línea de código; uno para explicar una línea en la misma línea.

66. a. Falso: es (PRINT (EVAL (READ))). b. Falso. c. Falso: de mayor profundidad. d. Falso: se asocia a las variables locales. e. Falso: solo CONS (y APPEND con un átomo al final). f. Falso: es una técnica de programación. g. Verdadero. h. Verdadero.

Banco, Unidad III

67. Corresponden al modelo de Von Neumann; tienen una línea simple de control de flujo; sus operaciones siguen un orden estricto; su comportamiento depende solo de las sentencias y de su orden; el tiempo de cada operación no influye en el resultado; su verificación es sencilla. Deterministas: con la misma entrada ejecutan la misma secuencia de instrucciones y producen el mismo resultado.

68. Ver la respuesta 1 del modelo C.

69. Ver la respuesta 1 del modelo C.

70. El concepto de hilo, que permitió mayor velocidad que con los procesos; los lenguajes orientados a objetos como Java, que incluyeron primitivas de soporte; y la aparición de Internet, cuyas aplicaciones se programan con técnicas de concurrencia.

71. Por la ganancia de procesamiento; por el incremento del throughput de las aplicaciones; para mejorar la respuesta de las aplicaciones hacia el usuario; porque es una estructura más apropiada para programas que controlan múltiples actividades y eventos; y porque da soporte a los sistemas distribuidos.

72. Ver la respuesta 2 del modelo A.

73. Paralelo se asocia con altas prestaciones en cómputo científico sobre una plataforma determinada (supercomputadores, clusters locales, en una misma sala). Distribuido se refiere a aplicaciones que se ejecutan en múltiples computadoras, con recursos heterogéneos, que no tienen que estar en el mismo espacio físico. Un sistema distribuido es una colección de elementos de cómputo autónomos, enlazados por una red de interconexión y soportados por comunicaciones que permiten ver la colección como un sistema integrado.

74. Es el uso simultáneo de múltiples recursos de cómputo para resolver un problema: se usan múltiples procesadores, el problema se divide en partes que se resuelven concurrentemente, cada parte se divide en instrucciones que se ejecutan en diferentes procesadores, y hace falta un mecanismo de coordinación general.

75. Multiprogramación: los procesos comparten uno o más procesadores. Multiprocesamiento: cada proceso corre en su propio procesador, con memoria compartida. Procesamiento distribuido: cada proceso corre en su propio procesador, conectado a los otros por una red.

76. Tienen múltiples líneas de flujo de control; sus sentencias se ejecutan en un orden no estricto; la secuencialización se da entre puntos de sincronización; son un conjunto de procesos que colaboran y compiten. Para validarlos, el resultado debe ser independiente de los tiempos de ejecución y de la plataforma, y las operaciones solo pueden validarse individualmente si las variables no se actualizan concurrentemente.

77. Es el modelo más simple y natural para muchas aplicaciones; facilita el diseño orientado a objetos; permite compartir recursos; optimiza el uso de recursos en monoprocesadores; reduce tiempos de ejecución en multiprocesadores; facilita programas fiables mediante replicación.

78. Clasifica las arquitecturas según los flujos de instrucciones y de datos. SISD: una instrucción, un dato; la arquitectura secuencial. SIMD: un solo flujo de instrucciones sobre múltiples datos; procesamiento vectorial y matricial. MISD: múltiples instrucciones sobre el mismo dato; en la práctica no se construyó. MIMD: múltiples flujos de instrucciones sobre múltiples datos; paralelismo funcional y de datos.

79. Single Program, Multiple Data. Es una variante de MIMD: el mismo código se ejecuta en todos los procesadores con datos distribuidos, pudiendo seguir caminos diferentes en cada uno. Puede verse como una extensión de SIMD o una restricción de MIMD.

80. Memoria compartida (multiprocesadores): un único espacio de direcciones accesible a todos; los procesadores se comunican a través de la memoria común; ejemplo, SMP. Memoria distribuida (multicomputadores): cada procesador tiene su memoria local; no se comparte el espacio de direcciones; se comunican por paso de mensajes a través de una red de interconexión.

81. Las dos son de memoria compartida. UMA: sistema fuertemente acoplado, con el mismo tiempo de acceso a memoria para todos los procesadores. NUMA: sistema ligeramente acoplado, con la memoria repartida entre los procesadores; el acceso a la memoria no local es más lento.

82. Massive Parallel Processing: variante MIMD con memoria físicamente distribuida y una red de interconexión dedicada, de baja latencia y gran ancho de banda; puede tener desde unos pocos hasta miles de procesadores organizados en una topología (anillo, árbol, malla, hipercubo).

83. Alto rendimiento, eficiencia, escalabilidad, transparencia, portabilidad y tolerancia a fallos. Escalabilidad: capacidad de mantener o incrementar la eficiencia en proporción al tamaño del problema. Tolerancia a fallos: habilidad de recuperar el sistema desde fallos de sus componentes sin perder estabilidad.

84. Respecto de la ubicación, de la concurrencia y del paralelismo. La portabilidad de código se logra con estándares (OpenMP, MPI, Java) que permiten ejecutar en distintas arquitecturas; la de performance es más difícil: implica lograr un rendimiento similar en todas ellas.

85. Funcional: el cómputo se divide en subcálculos independientes, en fases (pipeline) o con un patrón jerárquico. Geométrica: se divide en secciones que corresponden a divisiones físicas o lógicas del sistema modelado. Iterativa: una o más operaciones se aplican repetidamente y de forma simultánea sobre los datos, en forma determinística o especulativa.

86. Ver la respuesta 2 del modelo C.

87. Consiste en especificar en qué procesador se ejecuta cada tarea, ya que normalmente hay más tareas que procesadores. Es complicado porque hay dos objetivos en tensión: ubicar tareas concurrentes en distintos procesadores mejora la concurrencia, pero ubicar tareas que se comunican mucho en el mismo procesador mejora la localidad.

88. Es disminuir el número de procesadores de un sistema paralelo, lo que en algunas ocasiones mejora la performance. Se piensa como asignar más de un procesador virtual a cada procesador físico; con P reales y N virtuales, N/P es el factor de aumento del cómputo en cada procesador físico.

89. Ver la respuesta 3 del modelo A.

90. Es una notación gráfica: un grafo dirigido acíclico en el que cada nodo representa un conjunto de instrucciones y una flecha de A a B indica que B solo puede ejecutarse cuando A haya finalizado. Los nodos en paralelo pueden ejecutarse concurrentemente.

91. Las instrucciones que pueden ejecutarse concurrentemente se escriben dentro del par cobegin/coend y pueden ejecutarse en cualquier orden; el resto del programa se ejecuta de manera secuencial.

92. Conjuntos: S1 lee {a, b} y escribe {p}; S2 lee {a, c} y escribe {q}; S3 lee {p, q} y escribe {r}; S4 lee {r} y escribe {s}; S5 lee {q} y escribe {t}. No cumplen: S1-S3 ({p}), S2-S3 ({q}), S2-S5 ({q}) y S3-S4 ({r}). Los otros seis pares sí.

	S1	**S2**	**S3**	**S4**	**S5**
S1	-	Sí	No	Sí	Sí
S2	-	-	No	Sí	No
S3	-	-	-	No	Sí
S4	-	-	-	-	Sí



```
begin
  cobegin
     p := a + b;
     q := a * c;
  coend
  cobegin
     begin
        r := p + q;
        s := r * 2;
     end
     t := q - 1;
  coend
end
```

93. a. Falso: son no deterministas. b. Falso: puede simularse sobre una sola CPU. c. Falso: Bernstein no restringe las lecturas compartidas. d. Verdadero. e. Verdadero. f. Falso: es acíclico.

Banco, temas fuera de las diapositivas 2026

94. Es una cláusula (disyunción de literales) con a lo sumo un literal positivo. Tipos: hecho (solo el literal positivo), regla (un literal positivo, la cabeza, y uno o más negativos, el cuerpo: `H :- B1, ..., Bn`) y objetivo o consulta (sin literal positivo).

95. La unificación es el proceso de encontrar una sustitución de variables que haga idénticos dos términos; así se empareja un objetivo con la cabeza de una cláusula. El backtracking (vuelta atrás) es el mecanismo por el cual, cuando un objetivo falla o se piden más soluciones, Prolog retrocede al último punto de elección y prueba la siguiente alternativa.

96. Es el árbol que representa todos los caminos de la resolución SLD para una consulta: la raíz es la consulta, cada nodo es el objetivo pendiente, cada rama corresponde a resolver el primer subobjetivo con una cláusula del programa, y las hojas son éxitos (objetivo vacío) o fallos. Prolog lo recorre en profundidad, de izquierda a derecha.

97. Un redex es una expresión reducible, de la forma `(λx.M) N`. La reducción beta reemplaza el redex por `M` con `x` sustituida por `N`. Una expresión está en forma normal cuando no contiene ningún redex, es decir, no puede reducirse más.

98. Que si una expresión se reduce por dos caminos distintos a dos expresiones, existe una tercera a la que ambas pueden reducirse. Consecuencia: si una expresión tiene forma normal, es única, sin importar el orden de reducción.

99. La sección crítica es el fragmento de código en el que un proceso accede a un recurso compartido. La exclusión mutua es la propiedad que garantiza que en cada instante un solo proceso esté dentro de su sección crítica. Un semáforo es una variable entera protegida, con dos operaciones atómicas (wait/P y signal/V), que se usa para lograr exclusión mutua y sincronizar procesos; si solo toma los valores 0 y 1 es un semáforo binario.

100. Es la situación en la que dos o más procesos quedan bloqueados indefinidamente, cada uno esperando un recurso que tiene otro. Para que ocurra deben darse a la vez cuatro condiciones: exclusión mutua, retención y espera, no apropiación y espera circular.